Muzz Chat

Android exercise — implementation decisions, assumptions, limitations and next steps · Pedro Loureiro · November 2026

A single chat screen, built with Jetpack Compose, where you chat with Sarah. Messages you send appear at once and are confirmed by a mock server. A "Reply as Sarah" button sends the typed text as her, so both sides can be tried on one device. Messages are stored in Room, survive restarts, and are paged as you scroll up. The repository has more detail: the README, `CONTEXT.md` (glossary), `docs/architecture.md`, `docs/api.md`, `docs/decisions.md`, `docs/design-system.md` and four ADRs.

1. Approach
2. Architecture and system modelling
3. Tech stack
4. API assumptions
5. Mobile assumptions
6. Implementation decisions
7. Limitations
8. What I would do with more time

1 Approach

I built this as if it were an isolated feature that would really be integrated into Muzz, and made it as production-ready as the exercise allowed. A chat screen that only appends rows to a local list would meet the brief. It would not survive contact with a real backend, though: messages arrive late, twice, out of order or not at all; history is longer than memory; and the device's clock and its sends can't be trusted to order a conversation.

So most of the time went into three things:

- **System design.** A server-authoritative message model, with a stable identity and a gap-free order.
- **Pagination and sync.** Cursor paging from Room, with a catch-up that finds and fills missed messages.
- **A mock backend that behaves like a real one.** Latency, an echo on an incoming stream, idempotent sends, a server-assigned order and history pages. The app is exercised through the same paths a real network would use.

It is not plug and play. A real integration would bring its own API, authentication, error cases and product rules. The contract the app relies on is written down (section 4), though, and the mock sits behind the same interfaces a real data source would implement, so integrating should be mostly a matter of replacing the data source rather than redesigning the feature.

I did ask myself whether this was over-engineering for an exercise. I decided I could not ignore the mechanisms a real chat needs in order to work. Where I left something out, such as failure handling, edits or a chat list, it is listed as a limitation, with the design for adding it. I also built a small design system from the Muzz press kit (palette, launcher icon, splash screen), so the result feels on brand rather than generic Material.

2 Architecture and system modelling

Layers and features

The app uses Clean Architecture inside each feature. The layers are packages in one Gradle module, and dependencies point towards the domain. There are two features. `chat` is the screen and its messages. `user` gives `chat` the current user. A feature may import only another feature's `domain`, and features depend one way (`chat → user`). A unit test, `ArchitectureTest`, scans imports and fails when a rule is broken, since packages are not checked by the compiler.

```
ui  ──>  domain  <──  data           di ──> ui, domain, data   (Koin)

chat/ui      ChatScreen, ChatViewModel, ChatItemMapper, stateless components
chat/domain  Message, ChatRepository, SendMessageUseCase, OtherUserDevice (mock)
chat/data    ChatRepositoryImpl, Room (messages, chat_sync), MessageRemoteMediator,
             ChatRemoteDataSource + MockChatRemoteDataSource, MockChatApi
user/domain  User, UserRepository             user/data  UserRepositoryImpl, mock remote
```

A use case exists only when an operation has logic of its own. `SendMessageUseCase` trims and validates the text and builds the message. Observing messages is forwarding only, so the ViewModel calls the repository directly. The repository coordinates a remote data source (the messaging service: send, history pages, one incoming stream) and a local one (Room, the single source of truth). `ChatRepositoryImpl` is the only place messages are written.

Domain model

Concept	Model
User	A stable UUID `id` and a `name`. The current user comes from `UserRepository.currentUser()`, backed by a `me()` call.
Chat	A conversation, identified by a server `chatId`. Every message belongs to one Chat, and every query and remote call is limited to one `chatId`.
Message	A sealed class. **Pending** is sent by this device and not yet stored by the server: it has the device time and no server id or order. **Confirmed** is stored by the server: it has a `serverMessageId`, a `seq` and the server timestamp. Both carry `chatId`, `senderId`, `clientMessageId`, `Content` (type + value; only `TEXT`) and `timestampMillis` (epoch millis, UTC).
Identity	`(senderId, clientMessageId)`. The device generates a UUID per message, and the server returns it on every copy, from every sender. That pair is the Room primary key, the UI list key and the server's idempotency key. A message keeps one key from pending to confirmed, so its bubble never jumps.
Order	`seq`: a per-Chat number the server assigns, starting at 1 with no holes. It alone orders confirmed messages, and pending ones come after them by device time. Timestamps are only for display and grouping. A missing `seq` means a missed message, and `seq` 1 is the start of the history.
Chat sync state	Stored per Chat, in the same transaction as the messages that move it: `oldestSeq` (where older history continues), `newestSeq` (newest `seq` with no gaps below it) and `reachedStart`.
ChatItem (UI)	A renderable row: a `Message` (direction, text, grouped, delivery status) or a `TimestampHeader`. Built by `ChatItemMapper`, pure Kotlin and written test-first.

Flows

```
Send     ViewModel → SendMessageUseCase (validate, build Pending) → Repository
         → insert pending row in Room (shown at once, faded with "sending" dots)
         → remote.sendMessage → response and echo on the stream carry the stored copy
         → first copy upserts by (senderId, clientMessageId): the row is confirmed in place
         → second copy writes the same values: no change

Receive  remote.incomingMessages() → same upsert, for the open Chat only
         → seq == newestSeq + 1 moves newestSeq on; a skipped seq is stored and starts a catch-up

History  Paging 3, Room as the PagingSource, a RemoteMediator per Chat (pages of 20)
         REFRESH: no sync state → newest page; otherwise catch up after newestSeq until hasMore = false
         APPEND (scroll up): page before oldestSeq, until the server says hasMore = false
         Paged query returns each message with its next neighbour, for the bubble tail

Render   PagingData.map → ChatItem.Message;  insertSeparators → TimestampHeader
         LazyColumn(reverseLayout = true), keyed by the message identity
```

Mock boundary

Everything that stands in for something the exercise does not have is marked `MOCK DATA` and isolated, so it can be deleted without touching the layers above:

- `MockChatApi` plays the server. It holds a 200-message seed from `assets/`, plus the messages sent to it, kept in a file so it never reuses a `seq` after a restart. `MockChatRemoteDataSource` adds ~250ms of latency and a `SharedFlow` in place of the WebSocket.
- `MockUserRemoteDataSource` answers `me()` with a fixed user.
- The chat id and Sarah's name and user id are constants beside `MainActivity`, passed as screen parameters where navigation arguments would come from.
- "Reply as Sarah" goes through `OtherUserDevice`, a mock contract that sends as Sarah through the mock server. Her message comes back on the incoming stream like one from any other device, so this device never writes it directly.

3 Tech stack

Choice	Why	Trade-off
Kotlin 2.3, coroutines and Flow	The standard for Android. Room, Paging and Compose all speak `Flow`, so the UI observes the database end to end with no adapters.	None worth noting for this scope.
Jetpack Compose (Material 3 as the base)	Declarative, stateless components that are quick to build and preview. The components are custom and read Muzz tokens, with Material used only for `Scaffold` and theming.	Custom components are more code than stock Material ones, but they match the brand and keep every state explicit.
Room 2.8 (with KSP)	The Jetpack database, with compile-time checked SQL, `Flow` queries, transactions for message and sync-state writes, and a ready `PagingSource`.	The generated paging source uses `LIMIT/OFFSET`, and it drops a change made during its first load, which needed a small wrapper.
Paging 3.5 with `RemoteMediator`	The standard way to page from a local database while fetching from the network, with placeholders, prefetch and `map` / `insertSeparators` for the display rules.	Steep learning curve, and transformations see one row at a time, so the query has to supply the next neighbour. `paging-common` reaches the domain layer (an accepted deviation).
Koin 4.2	Lightweight DI with a Kotlin DSL: no annotation processing, fast builds, little setup, and first-class Compose and ViewModel support with parameters (`chatId`, Sarah's id).	The dependency graph is resolved at run time, so a missing binding fails when the app runs, not at compile time as with Hilt/Dagger. For a large app, Hilt's compile-time checks would be worth its setup.
kotlinx.serialization	Kotlin-first and reflection-free through a compiler plugin. Used for the mock seed, the mock server's file and the DTOs, as a real API client would.	A different library from Moshi or Gson, if the main app uses one of those.
`java.time`, minSdk 26	Correct calendar-day and time-zone arithmetic for headers, with no desugaring and no `SimpleDateFormat`.	Android 7.x (API 24–25) is not supported.
JUnit 4, Robolectric, paging-testing	Every test runs on the JVM, including Room against a real SQLite in memory, so `./gradlew test` needs no emulator. `asSnapshot()` tests the paging wiring.	Robolectric's SQLite is not a device's, and there are no instrumented or UI tests.
One Gradle module, layers as packages	Fast to set up and build for two small features. `ArchitectureTest` enforces the import rules.	The compiler does not enforce boundaries, and build caching is per module. Splitting into modules is the first step in a larger codebase.
No navigation library	The app has one screen. The chat's arguments are passed where navigation arguments would come from.	Adding a chat list means adding Navigation Compose.
`core-splashscreen`	The same branded splash on API 26–30 as on 31+.	None.

4 API assumptions

The app is written against a backend contract the exercise does not have. The client relies on each guarantee below, so a real backend must provide it, or the client must change.

Area	Assumption
Authentication	Every call and the incoming stream carry the signed-in user's token. The server knows the caller from the token alone.
Sender	The server sets `senderId` from the token. An outgoing message has no sender field, so a device can send only as its own user.
Current user	`me()` returns the signed-in user, with the same id the server puts in `senderId`. The client compares the two to tell outgoing from incoming messages.
Send	`sendMessage(chatId, {clientMessageId, content, timestampMillis})` returns the stored message. The server assigns a random UUID `serverMessageId`, the Chat's next `seq` and its own timestamp.
Idempotency	`(senderId, clientMessageId)` is unique on the server. A resend returns the message already stored and stores nothing new. This is what makes retries safe.
Ordering	Each Chat numbers its messages by `seq`, from 1 with no holes, assigned in the same transaction that stores the message.
Immutability	Stored messages never change: no edits or deletes. A late or repeated copy therefore cannot overwrite newer state.
Incoming stream	One stream (a WebSocket) for every Chat the user belongs to. It includes the sender's own messages, as the echo that confirms them. Every copy carries `clientMessageId`. Delivery order does not matter, and the stream may miss messages.
History	`fetchMessagesBefore(chatId, beforeSeq?, limit)` returns older messages (the newest page when there is no cursor). `fetchMessagesAfter(chatId, afterSeq, limit)` returns newer ones. Each returns `{messages, nextCursor, hasMore}`. `hasMore = false` on an older page means the start of the Chat.
Time	Timestamps are epoch milliseconds, so always UTC, and are used only for display and grouping.
Left out	Network errors, timeouts, 4xx/5xx, rate limits, payload limits, reconnection, token refresh and sign-out. The mock is always reachable and always succeeds.

5 Mobile assumptions

What the chat screen expects from the rest of a real Muzz client, which this exercise does not include:

Area	Assumption
Navigation into a Chat	The screen that opens a Chat, such as a chat list or a match, passes the Chat's `chatId`, the other user's id and their display name as navigation arguments. The chat screen shows the name in the top bar at once, with no extra request. Here `MainActivity` passes mock constants in their place.
Session	A user is already signed in, with a valid token, before the chat opens. Sign-in, token refresh and sign-out belong to another part of the app. The current user is read from `UserRepository` and not cached.

One open Chat	Incoming messages are stored for the Chat that is open, while its ViewModel is alive. Other Chats' messages are fetched when they are opened, through the catch-up. An app-level sync would take this over once a chat list exists.
Device clock	The device time is used only for a pending message, until the server's time replaces it. A wrong clock cannot reorder the conversation.
Time zone and locale	Header labels use the device's time zone and locale, read each time the list changes.
Local data	Room is a cache of the server's history and can be rebuilt from it. The app has never been released, so schema changes need no migrations yet.
Platform	Android 8.0+ (minSdk 26, for `java.time` without desugaring), phones in portrait and landscape, English, light theme.
Configuration changes and process death	The typed text lives in the ViewModel, so it survives rotation. Messages live in Room, so they survive the app being killed. A send in flight is tied to the ViewModel (see Limitations).

6 Implementation decisions

The brief's rules

Rule	How it is applied
Newest at the bottom; sent right, received left, in bubbles	A `LazyColumn` with `reverseLayout`. Direction comes from comparing `senderId` with the current user's id at mapping time, never from the code path that stored the message.
Small spacing: same sender and < 20s apart	The message is *grouped*: 2dp below it instead of 8dp, and no bubble tail, so the tail marks the last bubble of a run. The paged query supplies the next message, because paging maps one row at a time.
Section header: previous message > 1h older, or none	Applied with `insertSeparators`. The label depends on how long ago the message was: today `11:59`, within 6 days `Thursday 11:59`, older `12 Mar 2026 11:59`. A bare weekday would be ambiguous for older history.
Trigger the other user's messages	A "Reply as Sarah" button beside Send, sending through the mock server, as described in Mock boundary.
Bonus: persistent DB observed by the UI	Room is the only source of truth, and the UI observes it through `Flow<PagingData>`, with no copy of the messages held anywhere else.

How message identity and order evolved

Message identity was the hardest part to get right, and it changed three times as I tested each version against what a real server and a second device would do:

Version	Design	Why it changed
1. Device id, status flip	The device generated the message id and timestamp. The echo flipped the stored message from `SENDING` to `SENT`.	The device owned identity and order, so a wrong clock or a second device could reorder the conversation. A real server owns both.
2. Server UUIDv7	The server generated a time-ordered UUIDv7 that both ordered and paged messages. A pending message was stored under a device `localId`, and the echo replaced it: a delete and an insert in one transaction.	UUIDv7 values are not consecutive, so the app could not tell whether a message between two ids was missing. Same-millisecond ties were arbitrary. Messages from other devices had no `localId`, so the UI key changed source, and confirming took two writes.
3. Per-Chat `seq`	A server-assigned sequence with no holes orders and pages each Chat. A sync state per Chat and a forward catch-up fill gaps exactly.	Ordering was solved, but the pending/confirmed swap and the inconsistent UI key remained.
4. `(senderId, clientMessageId)` + `seq` (current)	The server returns the sender's `clientMessageId` on every message. That pair is the key everywhere, and the first server copy confirms the pending row in place with one upsert. `serverMessageId` is just an opaque id, and `seq` is just the order.	—

Trade-offs of the final design. Every message has one key from the moment it is sent, on every device. Confirming is one idempotent write, retries are safe, and a missing message is detected exactly. The costs:

- The server must assign `seq` atomically per Chat, which serialises writes within a Chat. That is fine for one-to-one chats, but busy group chats would need batching or sharding.
- A deleted message must keep its `seq`.
- The client keeps more state (a sync row per Chat).
- Each revision cost time in docs, code and tests. Getting identity right early made paging, echoes and the scrolling behaviour straightforward afterwards.

Main decisions

Decision	Why
Optimistic send, confirmed in place by the first server copy	Instant feedback, one write per confirmation, and the bubble never jumps. A repeated echo is harmless by construction. (ADR-0001)
Server-assigned per-Chat `seq`, with no holes	Missed messages are detected exactly, and the start of the history is known. A time-ordered UUID or a timestamp can only guess at gaps. (ADR-0003)
Paging 3 with Room as the source and a `RemoteMediator`	History of any length, fetched by cursor so new messages never cause repeated or skipped rows. A forward catch-up on open fills gaps without dropping stored history. (ADR-0003)
Clean Architecture as packages, with a test enforcing imports	Clear boundaries without the build setup of separate modules for two small features. (ADR-0002, ADR-0004)
Use cases only where there is logic	Avoids use cases that only forward a call. The ViewModel calls the repository for observing messages.
A separate `user` feature	Removes the global "me" constant. `chat` asks `UserRepository` for the current user, as it would with a real session. (ADR-0004)
Stateless composables	The screen owns all state, including scroll, press and focus. Components only render what they receive.
Two-tier design tokens, Muzz palette	Components read semantic roles, never raw values, which also makes dark mode a palette change. The launcher icon and splash screen are branded.
Deliberate visual departures from the brief	The input is filled instead of outlined, sending shows pulsing dots instead of ticks (the app knows only "sending" and "sent"), and there is no overflow menu (no action to put in it).
Scrolling	New messages scroll into view only when you are at the bottom. Reading older history is never interrupted, and Jump to latest appears instead. Sending snaps to the newest message, so what you send always comes into view.
Room paging workaround	`FirstLoadSafePagingSource` re-invalidates when the table changed during the paging source's first load. Room 2.8.5 drops such a change, which left a confirmed message shown as sending.
Tests	Logic is written test-first: the mapper, use case, ViewModel, repository, mediator, mappers, converters and date utils. The DAO and paged query run on an in-memory Room database with Robolectric. Composables are not tested.

7 Limitations

- **No failure paths.** The mock never fails, so a failed send would stay "sending" with no retry. The UI has no error or connection state, and shows loading only for a Chat's first load, on a fresh install.
- **Sends are tied to the screen.** If the ViewModel is cleared before the server answers, the message stays pending for good. Nothing resends it.
- **Outgoing status is only "sending" and "sent".** There are no delivered or seen states and no sent or seen times.

- **Text only**, with no edit, delete, reply, or clearing the chat. To start over, clear the app's data.
- **One Chat, with no chat list.** Incoming messages are stored only while the chat is open. There are no notifications.
- **Header labels update only when messages change.** A `11:59` label shown across midnight stays until the next message arrives.
- **No memory or storage cap.** Pages scrolled past stay in memory, and Room keeps every page fetched.
- **The current user is not cached.** With a real backend, every send and list open would wait for `me()`.
- **Light theme and English only.** White on Muzz pink is about 3.6:1, below WCAG AA for body text, kept for the brand.
- **No Room schema history or migrations.** The app was never released, so the schema changes in place.
- **Layers are packages, not modules.** Boundaries are checked by a test, not the compiler.

8 What I would do with more time

Product features

Feature	Approach
Sent at and seen at times	Show the server time of each outgoing message, on the last bubble of a run or on tap. Show the seen time from the other user's read receipt.
Delivered and seen states	The server keeps a `deliveredSeq` and `readSeq` per member of a Chat, and sends a receipt event when one moves. Because `seq` has no holes, one number per Chat marks every message up to it, so no per-message receipts are needed. The status follows from `seq ≤ readSeq`, rendered as ticks.
Network errors and retries	Pending rows become an outbox, sent from a WorkManager job with a network constraint and exponential backoff, so a send outlives the screen. Network errors and 5xx are retried with the same `clientMessageId`, which is safe through server idempotency. A 4xx response, or running out of retries, marks the message `FAILED` with tap to retry.
Delete messages	A tombstone: the message keeps its `seq` and loses its content, with a `deleted` flag, so it never becomes a hole. Delivered as its own stream event.
Edit messages	`editedAt` plus a server `changeSeq` that rises with every change. A change is applied only when it is newer than the stored one, so a late event cannot overwrite a newer edit. An "edited" label on the bubble.
Simple account information for Sarah	Tapping the top bar opens a profile sheet: photo, name, age, location and a short bio, from a `user(id)` call in the `user` feature. The avatar shows her photo instead of an initial.
Reply to a specific message	A message carries the replied-to message's identity, and the server returns a small preview. The bubble shows a quote, and tapping it scrolls to the original by `seq`, fetching its page if needed. Swipe to reply.
Dark mode	Components already read semantic roles only, so this means a dark mapping of tier-2 tokens to primitives and following the system setting, then checking contrast.
Images, documents, videos	New `ContentType` values with attachment metadata (URL, MIME type, size, dimensions, thumbnail). Upload to media storage first, through a pre-signed URL from a WorkManager job, then send the message. Pending bubbles show upload progress, and images load with Coil.
Audio messages	Hold to record, with a cancel gesture. Uploaded like other media, with a duration and waveform in the metadata, and played inline with Media3.

Engineering follow-ups

Each one is already described step by step in `docs/decisions.md`.

- **Chat list and app-level sync.** One handler for the stream that follows the app's foreground and stores every Chat. A `chats` table with the last message, activity time and unread count, a catch-up after every reconnect, and push notifications.
- **Error and connection state, and loading for older pages.** Render the paged list's `LoadState`, and add connection state to `ChatUiState`.
- **Room migrations.** Export the schema, version every change, and test each migration with `MigrationTestHelper`.
- **Localisation.** Add translations and locale-aware date patterns, and check layouts with longer text and in right-to-left languages.

- **Gradle modules.** Split each feature into domain, data and presentation modules, so the compiler enforces boundaries.
- **Keep header labels current.** Invalidate the list at midnight and on time zone or locale changes.
- **Long histories.** Cap pages in memory, trim old messages from Room, and replace the generated paging source with one keyed by `seq`.
- **Cache the current user.** Keep the signed-in user on disk from sign-in to sign-out.